

AULA 6

Laboratório: Persistência de Dados

Disciplina: CK0207 — Desenvolvimento de Software para Web

Professor: Dr. Fernando Antônio Mota Trinta

Universidade Federal do Ceará (UFC)

Projeto do curso: `nest-api-curso` (continuação da Aula 5)

Autor: Samuel Sales Furtado

Objetivos de Aprendizagem

- Conectar o Prisma Client dentro do NestJS através de um PrismaService
- Trocar o array em memória do UsersService por consultas reais ao banco
- Implementar create, findMany e findUnique
- Implementar update e delete usando o Prisma Client

Na Aula 5, o `schema.prisma` foi modelado e a primeira migration foi aplicada — o banco já existe, com a tabela `User` criada. Faltava conectar isso dentro do código do NestJS. Esta aula fecha esse ciclo: ao final, reiniciar o servidor não apaga mais os usuários cadastrados.

1. Conectando o Prisma ao NestJS

A peça central desta aula é o `PrismaService`: encapsular o `PrismaClient` dentro de uma classe própria, gerida pelo container de injeção de dependências do Nest — do mesmo jeito que qualquer outro service do curso.

```
import { Injectable, OnModuleInit, OnModuleDestroy } from '@nestjs/common';
import { PrismaClient } from '../../generated/prisma/client';
import { PrismaBetterSqlite3 } from '@prisma/adapters-better-sqlite3';

@Injectable()
export class PrismaService
  extends PrismaClient
  implements OnModuleInit, OnModuleDestroy
{
  constructor() {
    const adapter = new PrismaBetterSqlite3({
      url: process.env.DATABASE_URL || 'file:./dev.db',
    });
    super({ adapter });
  }

  async onModuleInit() {
    await this.$connect();
  }

  async onModuleDestroy() {
    await this.$disconnect();
  }
}
```

- `extends PrismaClient` — a classe herda todos os métodos de consulta do Prisma.
- **Driver adapter** (`PrismaBetterSqlite3`) — obrigatório na versão atual do Prisma, mesmo para SQLite.
- `onModuleInit()` / `onModuleDestroy()` — conectam e desconectam o banco acompanhando o ciclo de vida da própria aplicação NestJS.
- `|| 'file:./dev.db'` — valor de segurança, caso a variável de ambiente não seja carregada.

Atenção — três erros comuns: (1) o `schema.prisma` da Aula 5 precisa ter `moduleFormat = "cjs"` no bloco `generator client` — sem essa linha, o Prisma gera o client em formato ESM, e o Node quebra ao carregá-lo dentro de um projeto NestJS padrão (que roda em CommonJS), com o erro `ReferenceError: exports is not defined in ES module scope`. Se o client já tiver sido gerado sem essa linha, adicione-a, apague a pasta `generated/` e rode `npx prisma generate` de novo. (2) o caminho `'../../generated/prisma/client'` tem **dois** `../`, porque `prisma.service.ts` fica em `src/prisma/`, dois níveis abaixo da raiz onde a pasta `generated/` foi criada. (3) o NestJS não carrega o `.env` automaticamente — é necessário instalar `dotenv` e adicionar `import 'dotenv/config';` como a **primeira linha** de `src/main.ts`, antes de qualquer outro import. Sem isso, `process.env.DATABASE_URL` chega `undefined` e o adapter quebra.

1.1 PrismaModule global

```
import { Global, Module } from '@nestjs/common';
import { PrismaService } from '../prisma.service';

@Global()
@Module({
  providers: [PrismaService],
  exports: [PrismaService],
})
export class PrismaModule {}
```

Com `@Global()`, importar o `PrismaModule` **uma única vez** no `AppModule` disponibiliza o `PrismaService` para qualquer service do projeto, sem reimportar em cada módulo de feature.

2. Operações de Persistência

```
create(data: CreateUserDto) {
  return this.prisma.user.create({ data });
}

async update(id: number, data: UpdateUserDto) {
  await this.findOne(id); // garante 404 se não existir
  return this.prisma.user.update({ where: { id }, data });
}

async remove(id: number) {
  await this.findOne(id); // garante 404 se não existir
  await this.prisma.user.delete({ where: { id } });
}
```

A **assinatura** de cada método é a mesma desde a Aula 2/4 — só o corpo mudou. O Controller não muda nenhuma linha: continua chamando `userService.create()`, `.update()`, `.remove()`, sem saber se por trás existe um array ou um banco de verdade. É a separação de responsabilidades da Aula 1 rendendo o resultado mais concreto do curso até aqui.

Os métodos agora retornam `Promise`, já que toda operação do Prisma Client é assíncrona. O Nest lida com isso automaticamente nos handlers — na prática, isso significa usar `async / await` de forma consistente, inclusive no Controller (ver nota sobre `remove` mais abaixo).

3. Consultas ao Banco

```
findAll() {
  return this.prisma.user.findMany();
}

async findOne(id: number) {
  const user = await this.prisma.user.findUnique({ where: { id } });
  if (!user) {
    throw new NotFoundException(`Usuário ${id} não encontrado`);
  }
  return user;
}
```

`findMany()` lista tudo — equivalente ao antigo `return this.users`. `findUnique()` busca um só, por uma chave única, e **retorna null se não achar** — não lança erro sozinho. É por isso que o `if (!user) throw new NotFoundException(...)` continua necessário, exatamente como desde a Aula 4: a regra de tratamento de erro não mudou, só a fonte do dado.

Erro comum: se o Controller ou o Service não usarem `async / await / return` corretamente em cada chamada assíncrona, a resposta pode voltar `undefined` (virando `200` vazio em vez do esperado) ou a operação pode nem chegar a terminar antes do Nest responder. Sempre confira: o método é `async`, cada chamada ao `prisma.*` tem `await`, e o Controller tem `return` (ou `await + return`, no caso do `DELETE`).

Laboratório — Passo a Passo ## Passo 1 — Instalar dependências

```
npm install @prisma/adapter-better-sqlite3 dotenv
npm install --save-dev @types/better-sqlite3
```

Passo 2 — Carregar variáveis de ambiente Adicionar em `src/main.ts`, como primeira linha:

```
import 'dotenv/config';
```

Passo 3 — Criar o `PrismaService` e o `PrismaModule`

```
nest g module prisma
nest g service prisma
```

Usar o código apresentado na Seção 1 para `prisma.service.ts` e `prisma.module.ts` (lembrando do `@Global()`). ## Passo 4 — Importar o `PrismaModule` no `AppModule`

```
@Module({
  imports: [PrismaModule, UsersModule],
})
export class AppModule {}
```

Passo 5 — Reescrever o `users.service.ts` Usar o código das Seções 2 e 3, injetando `PrismaService` no construtor. ## Passo 6 — Testar com persistência de verdade

```
curl -X POST http://localhost:3000/users \
-H "Content-Type: application/json" \
-d '{"name":"Ana Souza","email":"ana@example.com"}'
```

Parar o servidor (`Ctrl+C`) e rodar de novo:

```
npm run start:dev
curl http://localhost:3000/users
```

Esperado: o usuário continua lá — diferente das aulas anteriores, onde os dados sumiam a cada reinício.

O que ainda falta

Próximo passo do curso: autenticação com JWT, na **Aula 9**, protegendo as rotas construídas até aqui.

Verificação de Conformidade

- ✓ `PrismaService` extends `PrismaClient` implements `OnModuleInit`, `OnModuleDestroy` — conferido em docs.nestjs.com/recipes/prisma.
- ✓ Driver adapter via `super({ adapter })` — conferido em documentação oficial do Prisma sobre integração com NestJS.
- ✓ `PrismaModule` com `@Global()` — padrão documentado oficialmente.
- ✓ `create`, `findMany`, `findUnique`, `update`, `delete` — conferido no Prisma Client Reference oficial.
- ⚠ **Necessidade de** `dotenv/config` **para carregar** `DATABASE_URL`, e caminho relativo correto do client gerado — pontos identificados durante a gravação real desta aula, incluídos aqui para evitar os mesmos erros.